Experiment 8: Add File Upload Functionality (Image or Text for Prediction)

Aim

To implement a user-friendly file upload feature in a Flask web app, allowing users to submit images or text files for machine learning prediction.

Software Requirements

- Python 3.10 or higher
- Flask
- Pillow (for image processing)
- scikit-learn
- numpy
- werkzeug
- HTML/CSS/JS for the frontend

Theory

File uploads enable users to provide external data for the server to process—commonly needed in real-world ML apps (e.g., upload a photo to recognize an object). Flask's `request.files` handles uploaded files, while libraries like Pillow allow image format handling and transformation before prediction. The workflow involves file validation, saving, preprocessing, then passing features into a trained ML model.

Procedure

Step 1: Create Project Directory

```
mkdir exp8_file_upload cd
exp8_file_upload
```

Step 2: Create and Activate Virtual Environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Step 2: Install Required Packages

```
pip install tensorflow pillow numpy flask werkzeug
```

Step 3: Directory Structure

```
mkdir templates static static/css static/uploads
```

Step 4: Train an Example Model

For demo, train a digit recognizer using scikit-learn's digits dataset: (**Save as train_digit_model.py**)

```
import numpy as np
from tensorflow.keras import layers, models from
tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# load and prepare MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data() x_train = x_train.astype("float32") /
255.0
x_test  = x_test.astype("float32")  / 255.0
x_train = np.expand_dims(x_train, -1)  # (N,28,28,1) x_test  =
np.expand_dims(x_test, -1)
```

```
y_train = to_categorical(y_train, 10) y_test  =
to_categorical(y_test, 10)

# small CNN
model = models.Sequential([
        layers.Conv2D(32, 3, activation="relu", input_shape=(28,28,1)), layers.MaxPool2D(2),
        layers.Conv2D(64, 3, activation="relu"), layers.MaxPool2D(2),
        layers.Flatten(),
        layers.Dense(128, activation="relu"), layers.Dense(10,
        activation="softmax")
])

model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"]) model.fit(x_train, y_train, epochs=5,
batch_size=128, validation_split=0.1) print("Test eval:", model.evaluate(x_test, y_test, verbose=0)) model.save("mnist_cnn.h5")
print("Saved mnist_cnn.h5")
```

Run with:

```
python train_digit_model.py
```

Step 5: Flask Backend (app.py)

```
from flask import Flask, request, jsonify, render_template from
werkzeug.utils import secure_filename
from PIL import Image, ImageOps
import numpy as np
import os
from tensorflow.keras.models import load_model
```

```
app = Flask(__name__)
app.config['UPLOAD_FOLDER'] =
'static/uploads'
os.makedirs(app.config['UPLOAD_FOLDER'],  exist_ok=True)

model = load_model("mnist_cnn.h5")

ALLOWED = {'png','jpg','jpeg','bmp'}

def allowed_file(name):
    return '.' in name and name.rsplit('.',1)[1].lower() in ALLOWED

def preprocess_image(path):
    # 1) load grayscale
    img = Image.open(path).convert('L')

    # 2) make square by padding preserving aspect ratio
    w,h = img.size
    sz = max(w,h)
    padded = Image.new('L', (sz,sz), color=255)  # white background
    padded.paste(img,  ((sz-w)//2,  (sz-h)//2))

    # 3) resize to 28x28
    small = padded.resize((28,28), Image.LANCZOS)

    # 4) invert if background is dark/bright mismatch arr =
    np.array(small).astype(np.float32)
    # Determine if digit is dark-on-light (common for photos) -> invert to MNIST style
(white digit on black)
    if arr.mean() > 127:
        arr = 255.0 - arr

    # 5) normalize to 0-1 and reshape
    arr = arr / 255.0
    arr = arr.reshape(1,28,28,1).astype(np.float32) return
    arr

@app.route('/')
def index():
    return render_template('upload.html')  # keep your existing upload.html

@app.route('/predict', methods=['POST'])
```

```
def predict():
    if 'file' not in request.files:
        return jsonify(error="no file"), 400 f =
    request.files['file']
    if not (f and allowed_file(f.filename)): return
        jsonify(error="bad file type"), 400
    filename = secure_filename(f.filename)
    path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
    f.save(path)

    X = preprocess_image(path)
    probs = model.predict(X)[0]
    pred = int(np.argmax(probs))
    conf = float(np.max(probs))
    return jsonify(prediction=pred, confidence=conf, filename=filename)

if __name__ == '__main__':
    app.run(debug=True)
```

Step 6: HTML Frontend (templates/upload.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>Digit Prediction Upload</title>
    <style>
        body { font-family: Arial; margin: 30px; background:#f5f5ff; }
        .container { background: white; border-radius:8px; padding: 40px; max-width: 450px; margin:auto; box-shadow: 0 5px 15px #aaa; }
    </style>
</head>
```

```
<body>
<div class="container">
    <h2>Upload Image for Digit Recognition</h2>
    <form id="uploadForm" enctype="multipart/form-data">
        <input type="file" name="file" accept="image/*" required><br><br>
        <button type="submit">Predict</button>
    </form>
    <div id="result"></div>
</div>
<script>
document.getElementById('uploadForm').onsubmit = async function(e) {
    e.preventDefault();
```

```
const form = e.target;
const formData = new FormData(form);
const res = await fetch('/predict', {method: 'POST', body: formData}); const
out = await res.json(); document.getElementById('result').innerHTML =
  out.prediction !== undefined
  ? '<p>Predicted Digit: <b>' + out.prediction + '</b></p><img src=\"/static/uploads/' + out.filename +
'\" width=\"100\">'
  : '<span style=\"color:red;\">' + out.error + '</span>';
};
</script>
</body>
</html>
```

Step 7: Run the App

```
python app.py
```

Step 8: Test

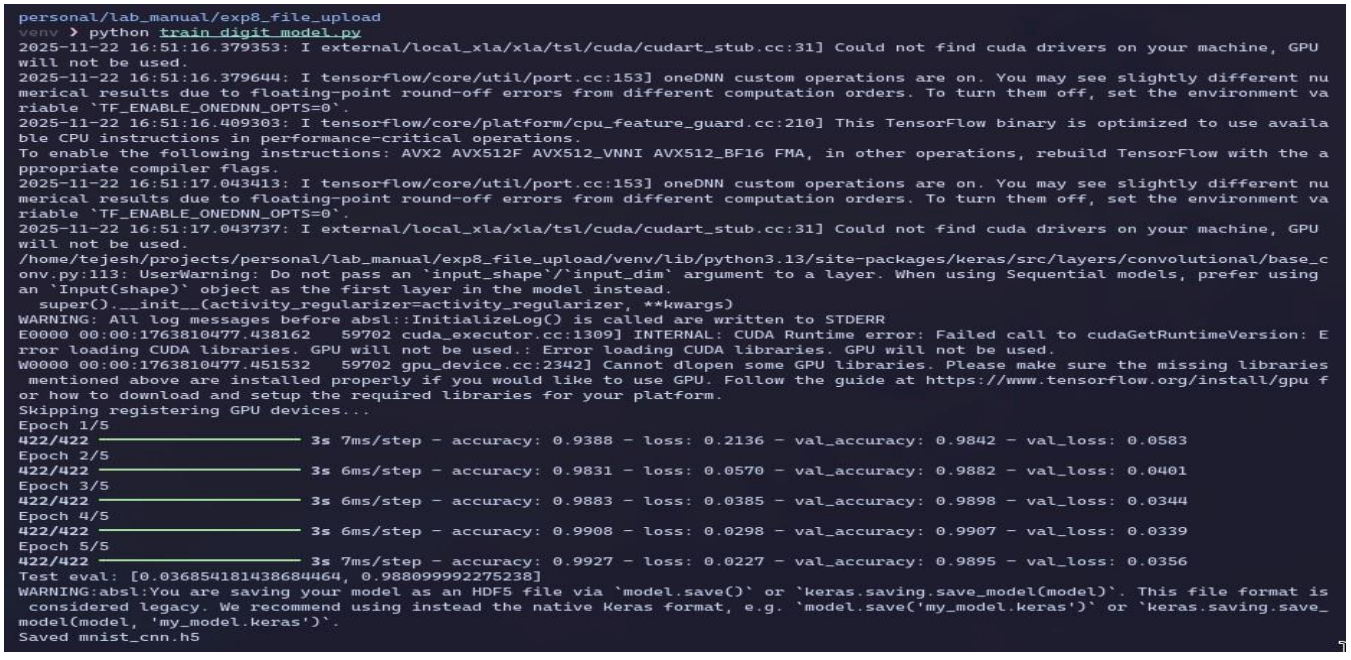
- Visit http://localhost:5000
- Upload a small image of a digit (8x8 or scaled)
- View the predicted digit

Output

- Predicted digit displayed below the form, with uploaded image preview.

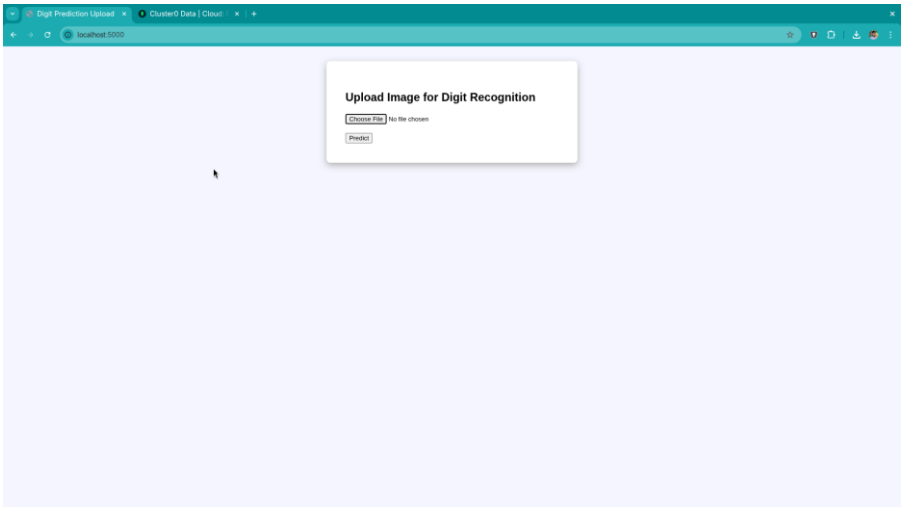
Screen-shots

Output of training model:

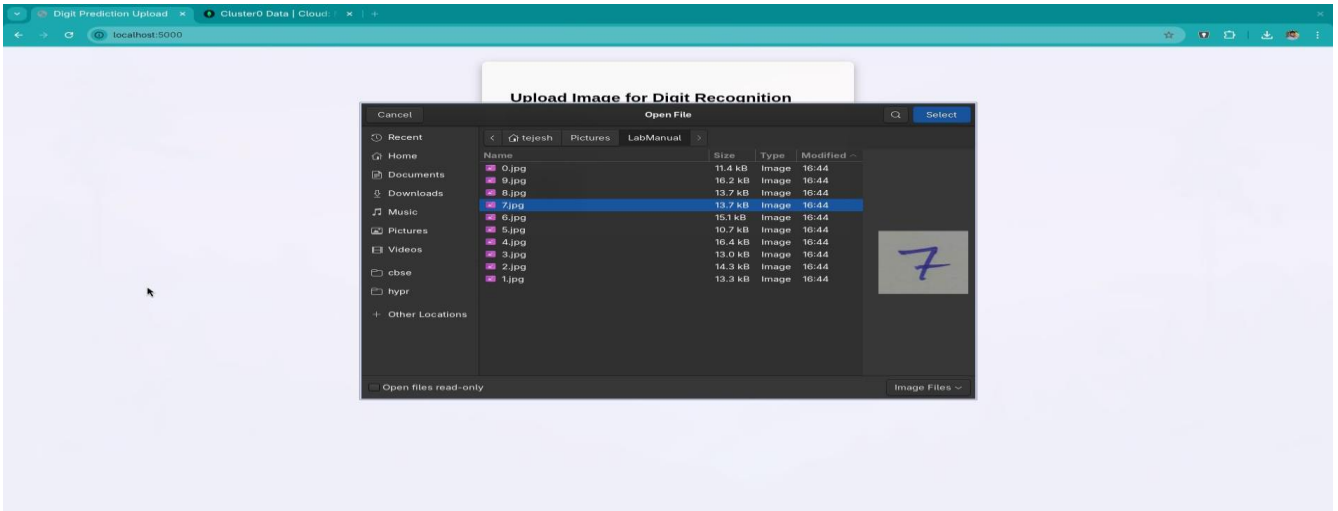


```
personal/lab_manual/exp8_file_upload
venv ❯ python train_digit_model.py
2025-11-22 16:51:16.379353: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU
will not be used.
2025-11-22 16:51:16.379644: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different nu
merical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment va
riable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-11-22 16:51:16.409303: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use availa
ble CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 FMA, in other operations, rebuild TensorFlow with the a
ppropriate compiler flags.
2025-11-22 16:51:17.043413: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different nu
merical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment va
riable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-11-22 16:51:17.043737: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU
will not be used.
/home/tejesh/projects/personal/lab_manual/exp8_file_upload/venv/lib/python3.13/site-packages/keras/src/layers/convolutional/base_c
onv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using
an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1763810477.438162   59702 cuda_executor.cc:1309] INTERNAL: CUDA Runtime error: Failed call to cudaGetRuntimeVersion: E
rror loading CUDA libraries. GPU will not be used.: Error loading CUDA libraries. GPU will not be used.
W0000 00:00:1763810477.451532   59702 gpu_device.cc:2342] Cannot dlopen some GPU libraries. Please make sure the missing libraries
 mentioned above are installed properly if you would like to use GPU. Follow the guide at https://www.tensorflow.org/install/gpu f
or how to download and setup the required libraries for your platform.
Skipping registering GPU devices...
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9388 - loss: 0.2136 - val_accuracy: 0.9842 - val_loss: 0.0583
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9831 - loss: 0.0570 - val_accuracy: 0.9882 - val_loss: 0.0401
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9883 - loss: 0.0385 - val_accuracy: 0.9898 - val_loss: 0.0344
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9908 - loss: 0.0298 - val_accuracy: 0.9907 - val_loss: 0.0339
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9927 - loss: 0.0227 - val_accuracy: 0.9895 - val_loss: 0.0356
Test eval: [0.036854181438684464, 0.988099992275238]
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is
 considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_
model(model, 'my_model.keras')`.
Saved mnist_cnn.h5
```

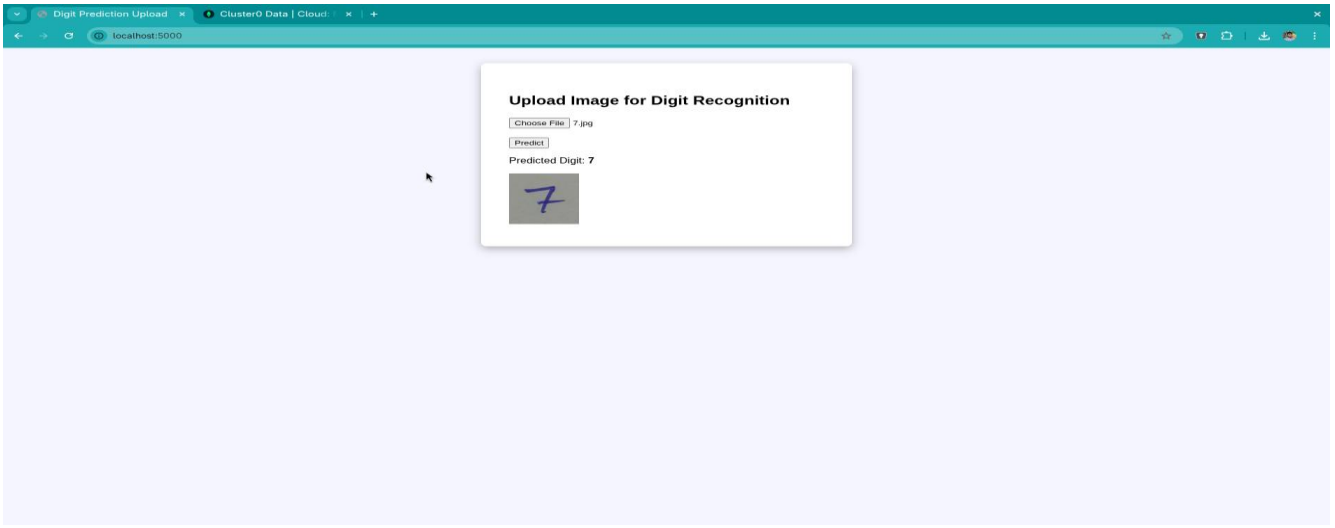
Default Page:



Selecting image



Model Prediction



> ⚠ **Warning**
>
> The above code has around 90% accuracy on external images. So it might predict wrong.